ResQCloud Project Report

Document status: Editable draft — replace every bracketed placeholder before submission.

Institution: [INSTITUTION / UNIVERSITY]

Department / Programme: [DEPARTMENT / PROGRAMME]

Course / Subject: [COURSE NAME AND CODE]

Project guide / supervisor: [NAME, TITLE]

Team: [TEAM NAME OR PROJECT GROUP]

Academic term: [SEMESTER / ACADEMIC YEAR]

Submission date: [DD MONTH YYYY]

Report version: [VERSION]

Repository: [REPOSITORY URL OR “PRIVATE REPOSITORY”]

Project site: https://dsl9l1s4fdu1c.cloudfront.net

AWS Certificates — Team Member 1

> This page is reserved for the AWS certificate evidence of one team member. Replace the fields and add that member's certificate image or PDF evidence. Keep one member per page. Duplicate this page for every additional member; remove unused member pages before submission. Do not include AWS passwords, access keys, MFA recovery codes, or other account credentials.

Certificate detail	Enter member information
Member's full name	[MEMBER 1 FULL NAME]
AWS certification / course	[CERTIFICATION OR COURSE NAME]
Issuing organization	[AMAZON WEB SERVICES / OTHER ISSUER]
Certificate or credential ID	[CREDENTIAL ID, IF PRESENT]
Issue / completion date	[DD MONTH YYYY]
Expiry date (if applicable)	[DD MONTH YYYY / NOT APPLICABLE]
Verification URL	[PUBLIC CREDENTIAL VERIFICATION URL]
Contribution to this project	[BRIEF ROLE OR CONTRIBUTION]

Certificate evidence: [INSERT CERTIFICATE IMAGE OR ATTACH THE MEMBER'S CERTIFICATE PDF HERE]

AWS Certificates — Team Member 2

Certificate detail	Enter member information
Member’s full name	[MEMBER 2 FULL NAME]
AWS certification / course	[CERTIFICATION OR COURSE NAME]
Issuing organization	[AMAZON WEB SERVICES / OTHER ISSUER]
Certificate or credential ID	[CREDENTIAL ID, IF PRESENT]
Issue / completion date	[DD MONTH YYYY]
Expiry date (if applicable)	[DD MONTH YYYY / NOT APPLICABLE]
Verification URL	[PUBLIC CREDENTIAL VERIFICATION URL]
Contribution to this project	[BRIEF ROLE OR CONTRIBUTION]

Certificate evidence: [INSERT CERTIFICATE IMAGE OR ATTACH THE MEMBER’S CERTIFICATE PDF HERE]

AWS Certificates — Team Member 3

Certificate detail	Enter member information
Member’s full name	[MEMBER 3 FULL NAME]
AWS certification / course	[CERTIFICATION OR COURSE NAME]
Issuing organization	[AMAZON WEB SERVICES / OTHER ISSUER]
Certificate or credential ID	[CREDENTIAL ID, IF PRESENT]
Issue / completion date	[DD MONTH YYYY]
Expiry date (if applicable)	[DD MONTH YYYY / NOT APPLICABLE]
Verification URL	[PUBLIC CREDENTIAL VERIFICATION URL]
Contribution to this project	[BRIEF ROLE OR CONTRIBUTION]

Certificate evidence: [INSERT CERTIFICATE IMAGE OR ATTACH THE MEMBER’S CERTIFICATE PDF HERE]

AWS Certificates — Team Member 4

Certificate detail	Enter member information
Member’s full name	[MEMBER 4 FULL NAME]
AWS certification / course	[CERTIFICATION OR COURSE NAME]
Issuing organization	[AMAZON WEB SERVICES / OTHER ISSUER]
Certificate or credential ID	[CREDENTIAL ID, IF PRESENT]
Issue / completion date	[DD MONTH YYYY]
Expiry date (if applicable)	[DD MONTH YYYY / NOT APPLICABLE]
Verification URL	[PUBLIC CREDENTIAL VERIFICATION URL]
Contribution to this project	[BRIEF ROLE OR CONTRIBUTION]

Certificate evidence: [INSERT CERTIFICATE IMAGE OR ATTACH THE MEMBER’S CERTIFICATE PDF HERE]

AWS Certificates — Team Member 5

Certificate detail	Enter member information
Member’s full name	[MEMBER 5 FULL NAME]
AWS certification / course	[CERTIFICATION OR COURSE NAME]
Issuing organization	[AMAZON WEB SERVICES / OTHER ISSUER]
Certificate or credential ID	[CREDENTIAL ID, IF PRESENT]
Issue / completion date	[DD MONTH YYYY]
Expiry date (if applicable)	[DD MONTH YYYY / NOT APPLICABLE]
Verification URL	[PUBLIC CREDENTIAL VERIFICATION URL]
Contribution to this project	[BRIEF ROLE OR CONTRIBUTION]

Certificate evidence: [INSERT CERTIFICATE IMAGE OR ATTACH THE MEMBER’S CERTIFICATE PDF HERE]

Abstract

ResQCloud is a web-based demonstration platform for recording and coordinating community incident reports. It supports a role-based workflow in which a citizen submits a report, a dispatcher reviews and assigns it, a responder records progress, and a supervisor closes the incident. Reporters can follow progress and provide feedback. The system includes an operational web interface, an API, persistent incident history, authentication and authorization, and optional AWS-backed storage, messaging, and notifications.

The project is implemented as a TypeScript monorepo. The web client uses Next.js; the API uses Express; shared Zod schemas and transition rules keep request contracts and incident-state behavior consistent. Local development uses SQLite, while the deployed AWS pilot uses Aurora PostgreSQL. The AWS deployment includes CloudFront, an application load balancer, ECS Fargate services, Amazon Cognito for citizen identity, private S3 evidence storage, SQS, SNS, CloudWatch, and Secrets Manager.

The system is deployed as a limited technical pilot at https://dsl9l1s4fdu1c.cloudfront.net. The deployment and basic endpoints have been checked. This report does not claim that the product is certified or ready for real emergency response. Use fictional data only until the documented security, privacy, operational, and jurisdictional readiness work is completed.

Keywords: incident management, emergency coordination demonstration, role-based access, Next.js, Express, AWS, Aurora PostgreSQL.

1. Project information

Item	Project information
Project title	ResQCloud — Community Incident Coordination Platform
Project category	[ACADEMIC / CAPSTONE / INTERNAL / OTHER]
Project owner	[ORGANIZATION / DEPARTMENT]
Project team	[NAMES AND STUDENT / EMPLOYEE IDS]
Project guide	[GUIDE NAME]
Repository	[REPOSITORY URL OR ACCESS DETAILS]
Production pilot URL	https://dsl9l1s4fdu1c.cloudfront.net
AWS Region	Asia Pacific (Sydney), `ap-southeast-2`
Report prepared on	[DD MONTH YYYY]
Demonstration data	Fictional data only

2. Description of the project

2.1 Background and problem statement

Incident reports can be difficult to coordinate when intake, triage, assignment, response updates, and closure are spread across separate channels. A single report record with explicit ownership and a history of status changes can make the work easier to follow. ResQCloud demonstrates this workflow in a web application with different workspaces for each role.

The project is a software engineering and cloud deployment demonstration. It is not connected to public emergency dispatch services. An urgent report creates an incident inside the application only; it does not call emergency services.

2.2 Purpose

The purpose of ResQCloud is to demonstrate an auditable, role-based incident workflow and the application architecture needed to support it. The project provides a working local demo and a limited AWS-hosted pilot for demonstration and evaluation.

2.3 Objectives

- Provide an interface for citizens to submit and track fictional incident reports.
- Provide operational workspaces for dispatchers, responders, supervisors, and administrators.
- Enforce role, ownership, and department authorization in the API.
- Record incident history and publish domain events reliably through a transactional outbox.
- Demonstrate a local SQLite workflow that does not require AWS.
- Demonstrate an AWS deployment using managed database, identity, storage, messaging, and hosting services.
- Document creation, local execution, deployment, verification, and known limitations.

2.4 Scope

Included in the current application are role-based incident intake and operations, assignments, status history, tracking, notifications, dashboards, administration functions, Cognito-hosted citizen sign-in, and optional AWS integrations. The production application uses Cognito for citizen identity and a separate staff/admin login path with password and multi-factor authentication. A production-only first-admin bootstrap path and admin-managed staff account creation are implemented in the deployed application.

The scope excludes live emergency-service dispatch, verified government/agency integrations, operational service guarantees, and authorization to process real sensitive data. Jurisdiction policy, legal/privacy review, disaster recovery, and several scale and security checks remain release gates.

2.5 Primary users and roles

Role	Main responsibilities
Citizen / reporter	Submit a report, retain the private tracking receipt, view status, and provide feedback after closure.
Dispatcher	Review and prioritize incoming reports, record triage decisions, and assign responders.
Responder	Accept an assignment, record progress, and resolve the assigned incident.
Supervisor	Review the outcome and close the incident.
Administrator	Manage authorized staff accounts and operational configuration and review audit information.

3. Steps involved in the incident workflow

1. **Report intake:** A citizen submits a fictional incident report and receives a tracking receipt.
2. **Triage:** A dispatcher reviews the report, confirms details and priority, and records the triage decision.
3. **Assignment:** The dispatcher assigns the incident to a responder.
4. **Response:** The responder accepts the assignment, records progress, and marks the response resolved.
5. **Supervisory closure:** A supervisor reviews the result and closes the incident.
6. **Follow-up:** The citizen checks the incident status and may provide feedback after closure.
7. **Audit and notification:** The system records state changes and can create in-app notifications and publish critical events through the configured event pipeline.

Every transition is validated by the API. Incident changes and their domain events are written transactionally so a committed state change can be delivered to downstream processing.

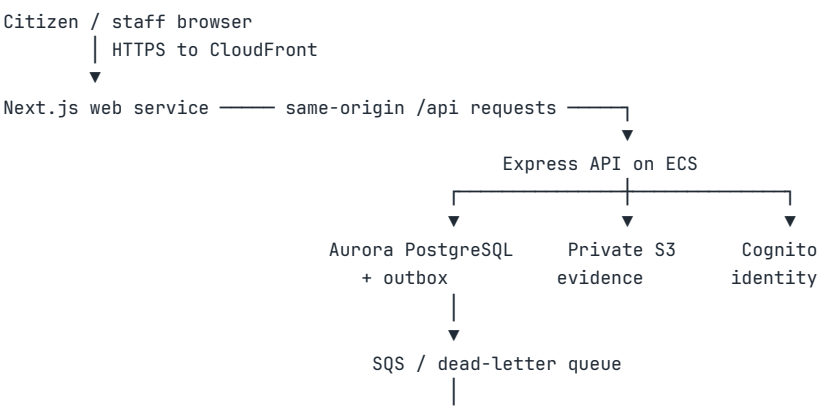
4. System architecture and technology

4.1 Application architecture

Layer	Technology / service	Responsibility
Browser interface	Next.js App Router, React, TypeScript	Role-aware portal, forms, dashboards, and incident workspaces.

Layer	Technology / service	Responsibility
Application API	Node.js, Express, TypeScript	Authentication, authorization, validation, business rules, and integration endpoints.
Shared contracts	TypeScript, Zod	Request/response schemas and shared validation rules.
Local database	SQLite	Convenient development and test database.
AWS database	Amazon Aurora PostgreSQL	Persistent pilot data, migrations, history, and transactional outbox.
Identity	Amazon Cognito plus application sessions	Hosted citizen sign-up/sign-in; separate authority/admin authentication and authorization.
Evidence storage	Private Amazon S3	Stores incident evidence objects. The bucket is not intended to be public.
Event delivery	Transactional outbox, Amazon SQS, dead-letter queue	Delivers incident events and supports retryable background processing.
Notifications	In-app records and Amazon SNS	Persists user notices and publishes configured critical-event notices.
Hosting and routing	CloudFront, Application Load Balancer, ECS Fargate	Serves the web application and routes API requests to the service.
Operations	CloudWatch, Secrets Manager, ECR	Logs application output, stores deployment secrets, and stores/scans container images.

4.2 High-level data flow





In the production pilot, CloudFront serves the application and routes `/api/*`, health/readiness endpoints, and socket traffic toward the API service. The API validates input, checks identity and authorization, then commits incident data and outbox events to PostgreSQL. A worker sends events to SQS when configured; consumers create idempotent in-app notifications and publish configured critical events to SNS.

4.3 Security and identity overview

- Citizen registration and sign-in use the Cognito hosted interface with authorization code flow and PKCE.
- The API validates the Cognito ID token before creating an application session.
- Staff and administrators use the separate work-account login with a password and authenticator code.
- Production self-service local registration is disabled; the first administrator is created through the one-time, protected bootstrap flow.
- Admin-created staff accounts require an MFA secret and current authenticator code. MFA secrets are encrypted before database storage.
- Passwords are hashed by the API. Production secrets are supplied through AWS Secrets Manager and task definitions rather than committed files.
- The API enforces authorization; hiding a page or button in the browser is not the security boundary.

The initial administrator must be provisioned by an authorized operator before staff administration is available. The bootstrap token is sensitive and must not be included in this report.

4.4 Production deployment boundary

The AWS pilot is deployed in `ap-southeast-2` and was last reported deployed and checked on 20 October 2026. The CloudFormation template and deployment script are maintained in `infra/aws/production.yaml` and `scripts/deployAwsProduction.mjs`. Basic homepage and health/readiness checks have passed. The AWS-hosted deployment is a limited technical pilot, not an operational emergency service.

Known limitations include single-task ECS services, HTTP between CloudFront and the load balancer origin, no rehearsed disaster-recovery process, no production CloudFront signing key for direct private-evidence delivery, incomplete privacy and policy approvals, and incomplete full-browser, load, security, and Aurora integration verification. The application should use fictional data only until these gates are reviewed and closed.

5. Steps for creation of the project

The following sequence describes how a developer can reproduce or continue the project from its repository. Replace the repository and institutional details in this report before submission.

5.1 Prepare the development environment

1. Install Git, Node.js 24 or newer, and npm. Python, `make`, and a C++ compiler may be needed to build the SQLite package on some systems.
2. Obtain repository access and clone the project:

   ```
   git clone [REPOSITORY URL]
   cd [REPOSITORY DIRECTORY]
   ```

3. Install the locked project dependencies:

   ```
   npm ci
   ```

4. Review the root `README.md`, `.env.example`, `docs/engineering/architecture.md`, `docs/engineering/security.md`, and `docs/project/status.md` before making changes.

5.2 Build the application modules

1. Define the shared incident, authentication, and API contracts in the shared package so the browser and API use the same validation rules.
2. Implement the API routes and domain logic in `apps/api`, including validation, authorization checks, database models/migrations, audit history, and error handling.
3. Implement the web routes and role-aware user interfaces in `apps/web`.
4. Add authentication and authorization paths for citizen, authority, and administrator roles.
5. Implement database transactions and the event outbox so incident state and domain events remain consistent.
6. Add external AWS integrations behind configuration: Aurora PostgreSQL, private S3, SQS, SNS, Cognito, and CloudFront as applicable.
7. Keep local development independent of the production account. The SQLite demo uses fictional records and does not require AWS credentials.
8. Document new settings and operational steps in the project documentation.

5.3 Configure and run the local project

Use the repository's demo launcher to prepare a separate SQLite database, fictional sample accounts, and local environment file:

```
npm run demo
```

Then open http://localhost:3000. The API runs on port `4000`; the demo launcher prints startup information. Use only the fictional development accounts and data. Stop both services with **Ctrl+C** in the terminal running the command.

5.4 Configure the AWS demonstration or pilot

For a local application connected to selected AWS services, follow `docs/aws/individual-setup.md` and `docs/aws/service-showcase.md`. Create a dedicated, MFA-protected AWS identity with the minimum needed permissions. Configure networking, TLS, database access, and each selected service before setting application variables. Never place AWS access keys or production secrets in browser variables or source control.

For the existing AWS-hosted pilot, use the documented CloudFormation stack and deployment process. Confirm the AWS account, Region, permissions, change set, database migration, and secret configuration before deploying. Production deployment changes AWS infrastructure and services; it should only be performed by an authorized operator following the project runbook.

6. Steps for execution of the project

6.1 Local execution

1. From the repository root, install dependencies with `npm ci` if needed.
2. Start the SQLite demo:

   ```
   npm run demo
   ```

3. Open http://localhost:3000.
4. Sign in with a provided development role or the fictional demo accounts listed in the repository README.
5. Submit a fictional citizen incident and save the demonstration tracking receipt.
6. Sign in as dispatcher and review, prioritize, and assign the incident.
7. Sign in as responder and record acceptance, progress, and resolution.
8. Sign in as supervisor to review and close the incident.
9. Return to the citizen view to follow status and provide post-closure feedback.
10. Review the application notifications and administrative audit functions as appropriate.
11. Stop the app with **Ctrl+C**. Use `npm run demo:reset` only when you intend to reset and remove the local demo database.

6.2 Production pilot execution

The deployed pilot is available at https://dsl9l1s4fdu1c.cloudfront.net. For a demonstration, verify current availability and use authorized accounts and fictional data. Citizens use Cognito sign-in. Staff/admin users use the work-account login with a password and current authenticator code.

Before staff administration can be used, an authorized operator must complete the first-admin setup at `/admin/bootstrap` using the bootstrap secret retrieved directly from AWS Secrets Manager. Do not copy that secret into the report, screenshots, email, source control, or chat. After the first admin exists, the bootstrap setup is closed and staff accounts are managed by an authenticated administrator.

6.3 Authorized deployment sequence

The repository documents this production sequence for an authorized deployment operator:

1. Review `docs/project/production-pilot.md`, the CloudFormation template, and the deployment script.
2. Verify the AWS CLI profile, intended account, Region, local prerequisites, and Docker/build support.
3. Run `npm run aws:preflight` to check deployment-host prerequisites.
4. Run `npm run aws:check-production-env` to audit the required environment and secret names in the live task definitions. This check does not display secret values.
5. Review the planned infrastructure changes and migration effects.
6. Run `npm run aws:deploy:production` only after the deployment is authorized and the reviewed target account is correct.
7. Allow the script to build and push the images, update the stack, run migrations/bootstrap work, update services, and wait for distribution propagation and health checks.
8. Confirm the application homepage and API `/health` and `/ready` endpoints, then record the deployment version and any failures.

The deployment script applies changes to AWS resources. Do not use production deployment commands as a routine local run step.

7. Verification and project results

7.1 Recorded implementation and deployment outcomes

- The API and web applications build successfully in the deployed revision.
- The Next.js application includes the citizen signup route, Cognito callback route, role-based workspaces, and administrator bootstrap page.
- Production uses Aurora PostgreSQL; local development uses SQLite.
- The production deployment script completed and reported the pilot online at the CloudFront URL.
- Live checks at deployment returned HTTP 200 for the homepage and signup page; the API health and readiness endpoints were also reported healthy.
- Production admin bootstrap status was checked and was still available at the time of the last recorded check. Verify it again before report submission or account provisioning.

These checks establish that selected paths and infrastructure were available at the time of those checks. They are not proof of full security, reliability, regulatory compliance, or readiness for real emergency operations.

7.2 Evidence to attach before submission

Evidence item	Placeholder / completion field
Local application home page	[INSERT SCREENSHOT AND CAPTION]
Citizen report submission and tracking receipt	[INSERT SCREENSHOT WITH FICTIONAL DATA ONLY]
Dispatcher incident review and assignment	[INSERT SCREENSHOT]
Responder progress / resolution	[INSERT SCREENSHOT]
Supervisor closure / audit history	[INSERT SCREENSHOT]

Evidence item	Placeholder / completion field
AWS architecture or CloudFormation overview	[INSERT DIAGRAM OR SCREENSHOT WITH ACCOUNT DETAILS REDACTED]
Build output and verification record	[INSERT DATE, COMMANDS, AND RESULT SUMMARY]
Live deployment verification	[INSERT DATE, URL, AND HTTP / HEALTH RESULTS]

Do not include private incident data, personal information, passwords, authenticator keys, tokens, AWS access keys, private IPs, or unredacted account-sensitive screenshots.

8. Project team and contribution record

Replace the rows below with the complete team roster. Keep contribution statements factual and confirm them with each member.

Member	Student / employee ID	Responsibility	Specific contribution	AWS certificate page
[MEMBER 1 NAME]	[ID]	[ROLE]	[CONTRIBUTION]	2
[MEMBER 2 NAME]	[ID]	[ROLE]	[CONTRIBUTION]	3
[MEMBER 3 NAME]	[ID]	[ROLE]	[CONTRIBUTION]	4
[MEMBER 4 NAME]	[ID]	[ROLE]	[CONTRIBUTION]	5
[MEMBER 5 NAME]	[ID]	[ROLE]	[CONTRIBUTION]	6

Additional members: [DUPLICATE AN AWS CERTIFICATE PAGE AND ADD A ROW FOR EACH ADDITIONAL MEMBER. UPDATE PAGE NUMBERS AFTER FINAL PAGINATION.]

9. Limitations and future work

The current deployment is a limited technical pilot. Before the system could be considered for a real operational setting, the project requires, at minimum:

- Review and approval of privacy, data retention, jurisdiction, agency participation, and incident handling policies.
- HTTPS for the full path from CloudFront through the load balancer to the services.
- Multi-task and distributed worker behavior, including WebSocket fan-out, rate limiting, and duplicate-work prevention.
- Verified backup restoration and a rehearsed disaster-recovery plan.

- Production security review, penetration testing, and role-complete browser testing.
- Aurora/PostgreSQL integration coverage, cloud failure-injection tests, and load/performance budgets.
- MFA recovery and account recovery procedures and a reviewed staff credential provisioning process.
- Malware scanning/quarantine and lifecycle cleanup for evidence files.
- Monitoring thresholds, alert ownership, support process, incident response, and documented service expectations.
- Verified jurisdiction data and agency authorization before any operational routing.

Until the relevant work and approvals are complete, the application is for demonstration with fictional data only and must not be relied on to contact or coordinate real emergency services.

10. Conclusion

ResQCloud demonstrates a complete software workflow for intake, triage, assignment, response updates, supervisory closure, and reporter follow-up. Its modular web/API architecture, shared validation contracts, transactional event handling, role-based authorization, local SQLite mode, and AWS pilot provide a useful base for continued academic and engineering work. The production deployment proves that a cloud-hosted demonstration is available; it does not certify emergency-response readiness. Further policy, security, operational, reliability, and jurisdiction work is required before the system is used with real incidents or sensitive data.

References and project materials

1. ResQCloud repository README: README.md
2. Application architecture: docs/engineering/architecture.md
3. AWS architecture: docs/aws/architecture.md
4. Production pilot configuration and deployment notes: docs/project/production-pilot.md
5. Project implementation status: docs/project/status.md
6. AWS individual setup: docs/aws/individual-setup.md
7. AWS service showcase: docs/aws/service-showcase.md
8. Project source repository: [INSERT REPOSITORY URL]
9. AWS documentation consulted: [INSERT AWS DOCUMENTATION REFERENCES REQUIRED BY YOUR COURSE]

Finalization checklist

- ☐ Replace every `[BRACKETED PLACEHOLDER]` with verified information or remove the field.
- ☐ Confirm the institution's report title page, citation style, font, margins, and page-number requirements.
- ☐ Confirm the complete team roster and duplicate/delete certificate pages so every member has exactly one separate page.
- ☐ Add each member's genuine AWS certificate evidence and verify certificate links/IDs.

- [ ] Confirm screenshots use fictional data and redact account-sensitive information.
- [ ] Recheck current deployment status and production admin-bootstrap status on the submission date.
- [ ] Update the project results with actual build/deployment verification dates.
- [ ] Review all claims with the project guide and team before submitting.